

Document Scanner & Enhancer

A command-line document scanner using classical computer vision:
edge detection, Hough transform, homography rectification and histogram-based
enhancement

Course	Computer Vision
Student name	[Your Name]
Registration number	[Your Reg. No.]
Faculty	[Faculty Name]
GitHub repository	https://github.com/[username]/doc-scanner-cv
Submission date	[dd-mm-yyyy]

Note: this is a faculty-prepared **reference sample** showing the expected structure, depth and evidence for the project report. Students must design, implement and document their own original project.

1. Introduction

Phones have replaced flatbed scanners for many everyday tasks, but a photograph of a page is not a scan. It is a perspective projection of a plane onto the image sensor: the rectangle of the page becomes a general quadrilateral, the background is included, and lighting varies across the page. Turning such a photograph into a clean, fronto-parallel image is a textbook computer-vision pipeline that touches image formation, low-level filtering, edge and line detection, and projective geometry.

This project implements that pipeline as a modular Python package with a command-line interface. The emphasis is on *understanding*: the Direct Linear Transform (DLT) homography and histogram equalisation are written from first principles with NumPy, and OpenCV is used for standard building blocks (Canny, Hough, resampling).

Syllabus coverage. Unit 1 (image formation, convolution and filtering, histogram processing, enhancement); Unit 2 (projective transformations, homography, DLT, rectification); Unit 3 (Canny edges, Hough transform, contour-based segmentation).

2. Problem Statement

Given a single photograph containing one roughly rectangular paper page against an arbitrary background, automatically (a) locate the page boundary, (b) remove the perspective distortion to produce a rectified page image, and (c) enhance the page so that text is legible despite shadows and uneven illumination. The system must run from a terminal, report measurable quality and accuracy figures, and fail gracefully on bad input.

Objectives

- Detect the page quadrilateral with sub-10-pixel corner error on controlled data.
- Estimate the homography with a from-scratch, numerically stable DLT.
- Improve page contrast and flatten illumination, with selectable colour / grey / black-and-white output.
- Provide an evaluation harness with ground truth so claims are measured, not assumed.
- Deliver a clean, tested, documented repository suitable for evaluation.

Target users: students and staff needing quick scans; CV learners; developers needing an OCR pre-processing stage. **Out of scope:** OCR, multi-page or curved pages, GUI front-ends, deep-learning models.

3. Functional Requirements

ID	Requirement	Module
FR1	Load and validate an input image (existence, extension, decodability, minimum size).	io_utils
FR2	Detect the page quadrilateral using edges and contours; fall back to Hough lines; as a last resort use the full frame.	Detection
FR3	Order corners and estimate a homography with a normalised DLT.	Rectification
FR4	Warp to a fronto-parallel image, optionally imposing an A4 or Letter aspect ratio.	Rectification
FR5	Enhance in three modes: color, gray, bw (illumination correction, CLAHE, sharpening, adaptive threshold).	Enhancement
FR6	Process one image, a folder of images, or a generated synthetic demo from the CLI.	main.py
FR7	Compute quality metrics (sharpness, contrast, entropy) before/after enhancement.	metrics
FR8	Evaluate on synthetic scenes against ground-truth corners, including a detector ablation; export CSV.	evaluate

Input/output structure. Input: JPG/PNG/BMP/TIFF image path (or folder). Output: *_scanned.png plus, with - -debug, edge map, detection overlay and rectified image; a log file; and evaluation.csv for the evaluate command.

User workflow. (1) Install dependencies; (2) run demo to see the system work; (3) run scan or batch on real photos choosing a mode and paper size; (4) inspect outputs and logs; (5) run evaluate to quantify accuracy.

4. Non-functional Requirements

Category	Requirement	How it is met / measured
Performance	Scan a 1.2 MP photo in well under one second on a laptop.	Detection runs on a copy downsampled to 1000 px. Measured about 60-70 ms per 1200x1000 image; about 0.8 s for a 12 MP image (development sandbox; hardware dependent).
Reliability	One bad file must never stop a batch.	Batch mode catches per-file errors, reports them, continues and returns exit code 2. Detection has three-level fallback (contour, Hough, full frame).
Usability	Self-explanatory CLI.	argparse sub-commands with help text, sensible defaults, and a <i>demo</i> command that needs no input data.
Maintainability	Modular, documented, testable code.	12 focused modules, a single validated config dataclass, no magic numbers, docstrings, 40 automated tests.
Error handling	Clear messages, correct exit codes.	Custom ImageLoadError; config validated at construction; CLI prints 'error: ...' and returns 1.
Logging	Traceable runs.	Console + file logging (logs/docscan.log), DEBUG with -v; warnings when fallbacks fire.
Resource efficiency	No heavy dependencies, bounded memory.	Only NumPy and OpenCV; processing resolution capped; peak memory about 300 MB even for a 12 MP input.

5. System Architecture

The system is a layered, pipeline-style architecture. The CLI layer parses arguments and handles errors; the orchestration layer (ScanPipeLine) sequences the three processing modules; supporting modules provide configuration, logging, I/O, metrics and evaluation. Each module has one responsibility and communicates through plain NumPy arrays and small dataclasses, which keeps coupling low and makes every module independently testable.

System Architecture (layered)

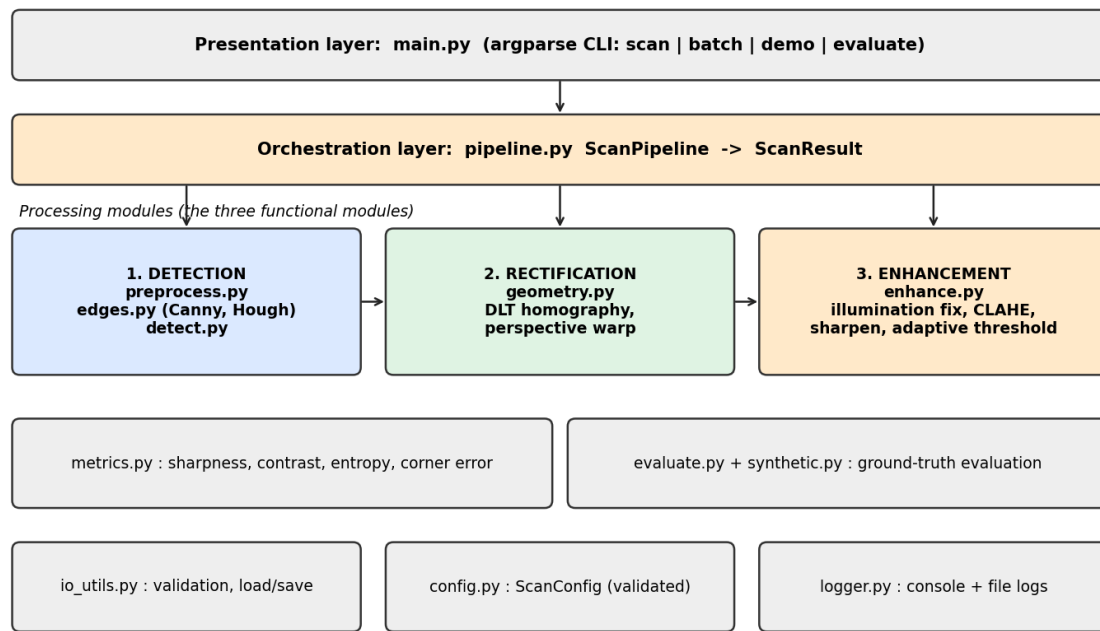


Figure 1 - Layered system architecture

6. Design Diagrams

6.1 Use Case Diagram

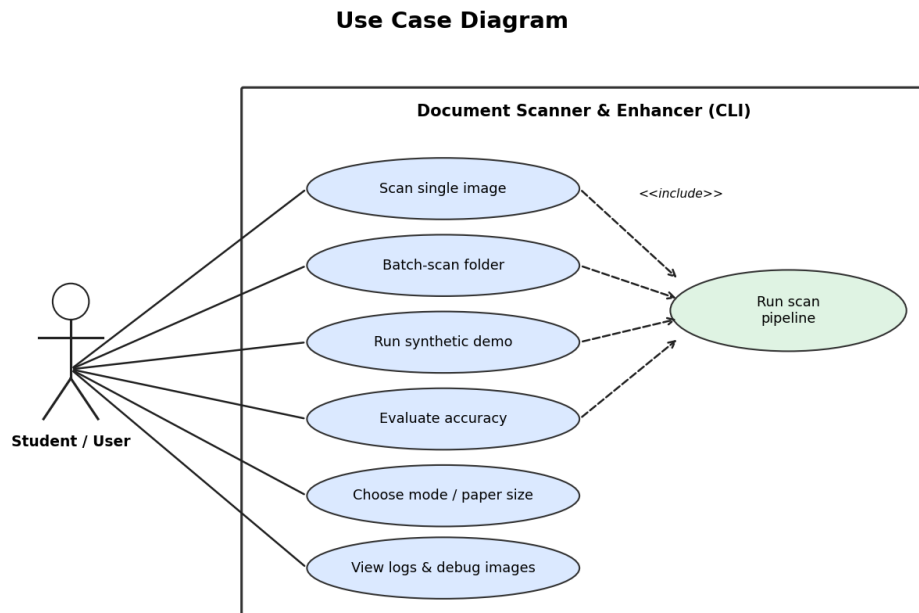


Figure 2 - Use cases for the student / user actor

6.2 Workflow Diagram

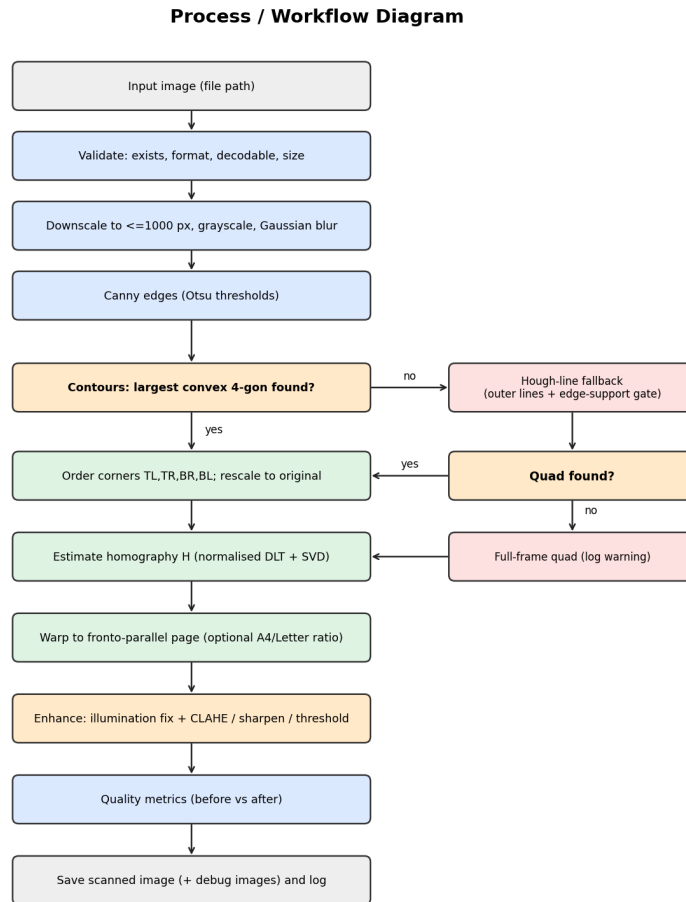


Figure 3 - Process flow including fallback strategies

6.3 Sequence Diagram

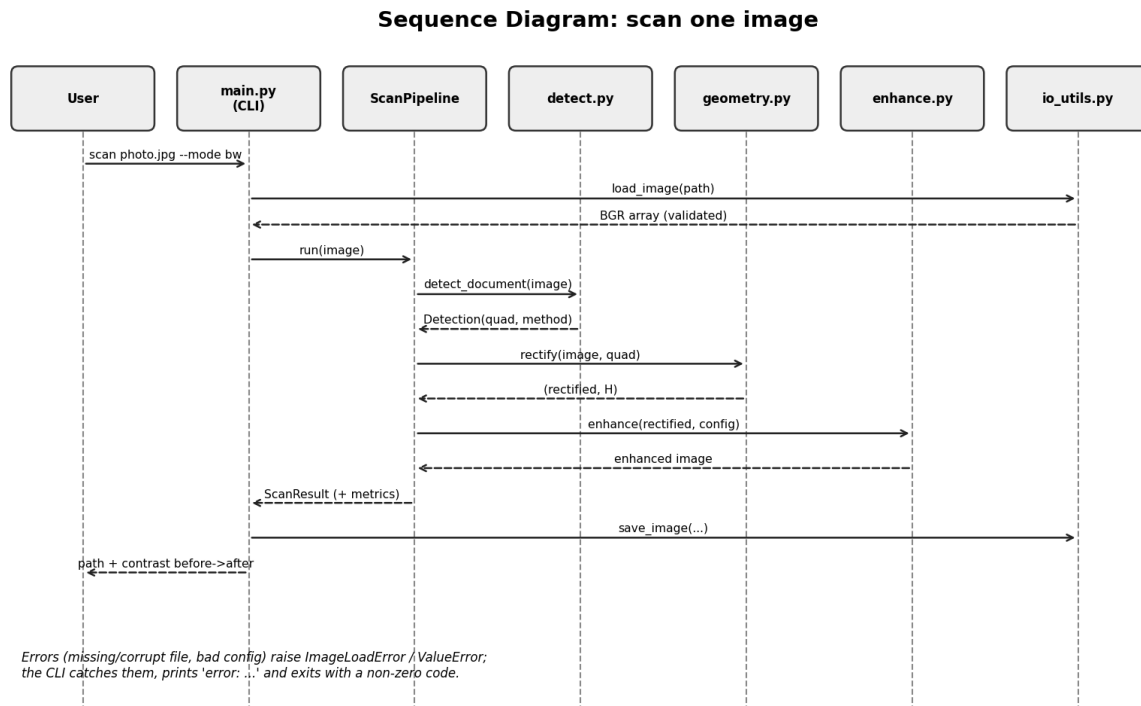


Figure 4 - Sequence of calls for the 'scan' command

6.4 Class / Component Diagram

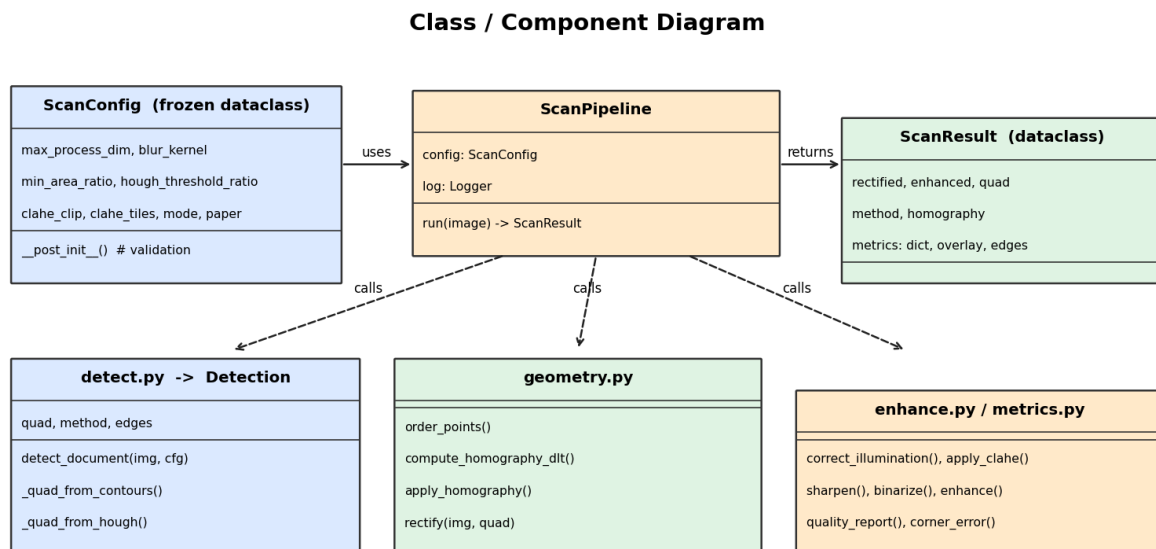


Figure 5 - Principal classes and module components

6.5 ER Diagram / Storage Design

Not applicable. The system keeps no database. Its only persistent data are image files and a CSV written to a user-chosen output folder, and a text log. The CSV schema is: scene, method, corner_error_px, success, hough_only_error_px, contrast_before, contrast_after, entropy_before, entropy_after.

7. Design Decisions & Rationale

Decision	Alternatives considered	Rationale
Contours as primary detector, Hough as fallback	Hough only; deep-learning segmentation	Contours on a closed edge map are robust to broken borders and give a direct 4-gon. Hough lines are useful but confused by text rules inside the page (see Challenges). Deep learning would need data and hide the maths.
From-scratch normalised DLT	Call <code>cv2.getPerspectiveTransform</code> only	Teaches the core multi-view-geometry idea. Hartley normalisation conditions the SVD; tests prove agreement with OpenCV.
Otsu-derived Canny thresholds	Fixed or median-based thresholds	Adapts to image brightness without manual tuning; low = 0.5 x high follows Canny's 1:2 guidance.
Detect on a downscaled copy, rectify at full resolution	Process everything at full size	Measured on a 12 MP image: detection takes about 60 ms downscaled versus about 110 ms at full resolution (1.8x faster). It also makes fixed kernel sizes behave consistently across camera resolutions, and output quality is kept because the warp uses the original pixels.
Flat-field correction before CLAHE	CLAHE alone	Shadows are a multiplicative background; dividing by an estimated background removes them, then CLAHE restores local contrast.
Optional paper aspect prior	Always use quad edge lengths	Edge lengths of a perspective-distorted quad do not give the true aspect ratio; a prior removes the stretch.
Frozen dataclass config with validation	Loose keyword arguments / globals	Fail fast on bad parameters; one place for every tunable value.
unittest instead of pytest	pytest	Zero extra dependencies; evaluators can run tests on any Python install.
Synthetic data with ground truth	Only real photos	Known corners allow numeric error measurement and a repeatable ablation; real photos are recommended as future work.

8. Implementation Details

8.1 Module 1 - Detection (`preprocess.py`, `edges.py`, `detect.py`)

The image is downscaled (longest side ≤ 1000 px), converted to grey and smoothed with a 5x5 Gaussian, which is a low-pass convolution that suppresses sensor noise before differentiation. **Canny** thresholds come from Otsu's method: high = T_{Otsu} , low = $0.5 T_{\text{Otsu}}$. The edge map is morphologically closed to bridge small gaps; `findContours` (external only) returns candidate blobs sorted by area. For each, the convex hull is simplified with `approxPolyDP` at $\epsilon = 2\%$ of the perimeter (relaxed to 3-5% if necessary) until exactly four vertices remain.

Hough fallback. If no 4-gon is found, the standard Hough transform returns lines (ρ , θ) with $\rho = x \cos(\theta) + y \sin(\theta)$. Lines within 30 degrees of horizontal and of vertical are grouped; the *outermost* line of each group is taken as a page border because text rules and table lines lie inside the page. Their four intersections give a candidate quad, which is accepted only if at least 45% of its perimeter lies on real edges (an *edge-support gate*) - a bad guess is better rejected than shown. If everything fails, the full frame is used and a warning is logged.

8.2 Module 2 - Rectification (geometry.py)

Corners are ordered top-left, top-right, bottom-right, bottom-left using the sum and difference of coordinates. A homography H maps source points x to destination points x' (in homogeneous coordinates, $x' \sim Hx$). Each correspondence gives two linear equations in the nine entries of H ; stacking them yields $Ah = 0$, solved as the right singular vector of A with the smallest singular value (SVD). Following Hartley, both point sets are first translated to zero mean and scaled to mean distance $\sqrt{2}$; the estimate is de-normalised as $H = T^{-1} H_n T$. The destination rectangle's size comes from the longest top/bottom and left/right edges (or from the A4/Letter ratio at equal area), and `warpPerspective` resamples with bicubic interpolation.

$$\begin{bmatrix} -x & -y & -1 & 0 & 0 & 0 & ux & uy & u \\ 0 & 0 & 0 & -x & -y & -1 & vx & vy & v \end{bmatrix} h = 0$$

Here (x, y) is a source point, (u, v) its destination, and h is the nine entries of H as a vector.

8.3 Module 3 - Enhancement (enhance.py, preprocess.py)

Illumination correction: dilation removes dark text, a large median blur (kernel = 8% of the image side) gives the page background $B(x,y)$, and the output is I/B scaled to 255, cancelling smooth shadows. **CLAHE** (clip 2.0, 8x8 tiles) then boosts local contrast. In *color* mode these operate on the L channel of CIELAB so colours are preserved; *gray* mode adds unsharp masking; *bw* uses an adaptive Gaussian threshold (block 31, $C = 12$). A global histogram equalisation, $s_k = \text{round}((\text{CDF}(k) - \text{CDF}_{\min}) / (N - \text{CDF}_{\min})) \times 255$, is implemented from scratch with a lookup table and verified against OpenCV.

8.4 Supporting modules

- **config.py** - frozen dataclass; invalid values raise `ValueError` at construction.
- **io_utils.py** - `ImageLoadError` for missing, unsupported, corrupt or tiny images; safe writing that creates folders.
- **metrics.py** - variance of Laplacian (sharpness), RMS contrast, Shannon entropy, mean corner error.
- **synthetic.py** - draws a text-and-table page, warps it with a random projective transform onto a textured table, adds a lighting gradient and noise; returns exact ground-truth corners.
- **evaluate.py** - runs N scenes, records errors, runs the Hough-only ablation, writes CSV.
- **main.py** - argparse CLI with sub-commands, error handling and exit codes (0 success, 1 error, 2 partial batch).

8.5 Version control

The repository was built incrementally with meaningful commits (scaffold, low-level modules, detection and pipeline, synthetic data, CLI, tests, diagrams, documentation), so the history documents the development process.

9. Screenshots / Results

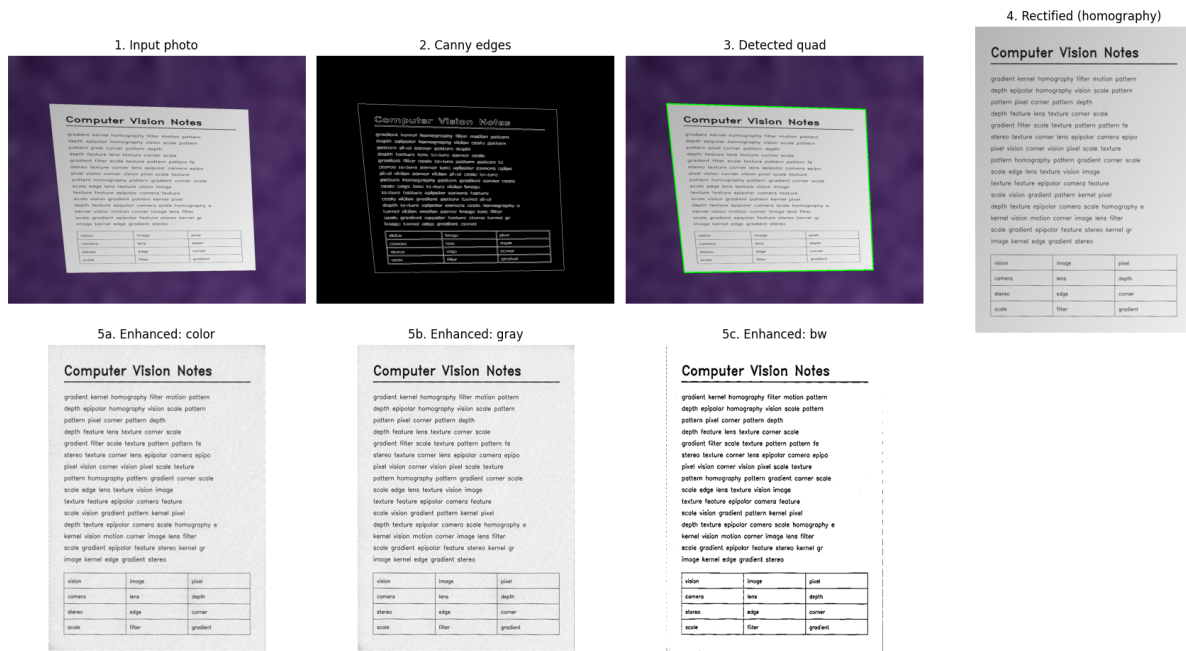


Figure 6 - Pipeline stages on a synthetic photo: input, edges, detected quad, rectified page and the three enhancement modes

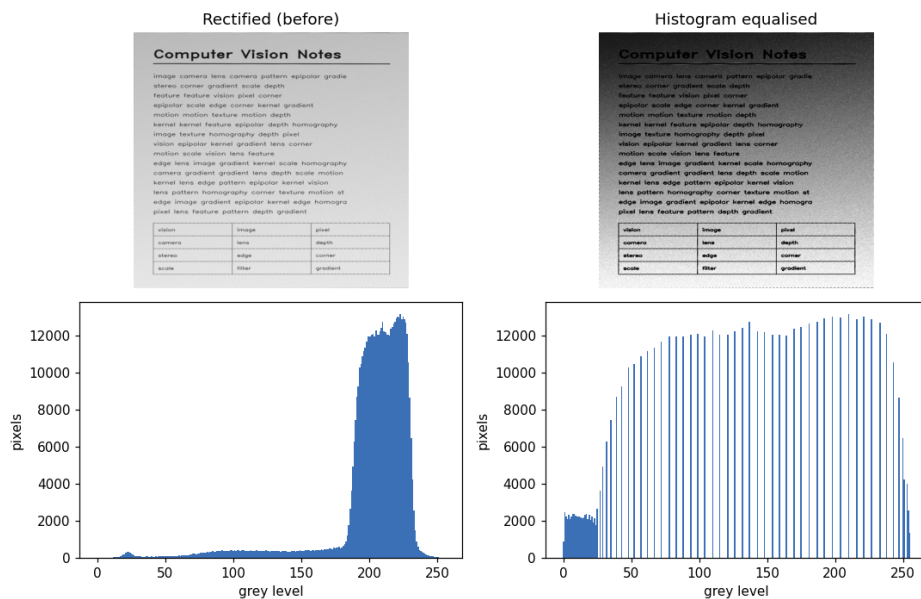


Figure 7 - Grey-level histogram before and after equalisation

Quantitative evaluation on 30 synthetic scenes (random perspective, lighting gradient and sensor noise):

Metric	Value
Detection success (mean corner error ≤ 10 px)	100%
Mean / maximum corner error (primary contour method)	0.73 px / 1.13 px
Hough-only fallback success (ablation)	90% (misses at 17.4, 28.4, 50.3 px)
Mean RMS contrast of rectified page, before to after enhancement (colour mode)	30.9 to 37.2
Mean grey-level entropy, before to after	6.20 to 5.89 bits
Average time per 1.2 MP image	about 60-70 ms

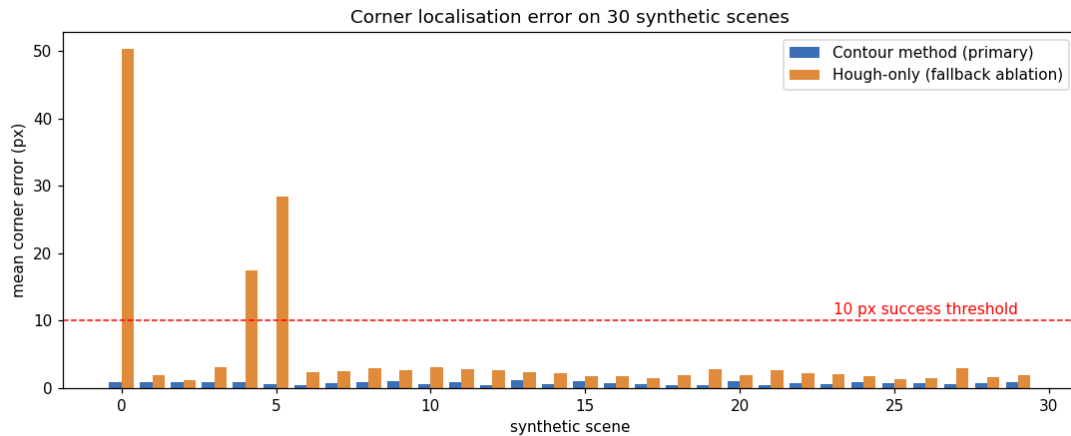


Figure 8 - Corner error per scene for the primary and fallback detectors

Interpretation. The contour method localises corners to about one pixel on controlled data, and the Hough fallback is a credible second opinion that fails on a minority of scenes. These figures come from *synthetic* scenes whose background is smooth and whose page contrast is high, so real-world accuracy will be lower; the numbers demonstrate that the implementation is correct, not that it is production-ready. Entropy *falls* after enhancement; this is expected and desirable for documents, because shadows and background gradients are replaced by a nearly uniform white page, leaving fewer distinct grey levels.

10. Testing Approach

The suite contains **40 automated tests** using Python's built-in unittest (`python -m unittest discover -s tests -t .`). Tests were written after the core modules and expanded as defects were found (notably the Hough fallback), and were re-run throughout development.

Test file	Tests	What it verifies
test_geometry.py	11	DLT maps points exactly; agrees with cv2.getPerspectiveTransform; identity case; noisy over-determined estimation; too-few and collinear points rejected; corner ordering under permutations; output size and aspect handling; end-to-end rectification of a known image.
test_preprocess.py	10	Histogram sums to pixel count; equalisation stretches contrast and is within 2 grey levels of OpenCV; flat image unchanged; resize helpers; illumination correction flattens a gradient; thresholded output is binary.
test_detect_pipeline.py	15	Contour accuracy on 5 scenes; Hough fallback accuracy and rejection of noise; blank image gives full-frame fallback; line intersection; all enhancement modes; contrast improvement; grayscale input; paper aspect; invalid input; config validation; IO errors; round-trip save/load; entropy bounds.
test_cli.py	4	Real subprocess runs: demo creates outputs; missing file returns exit code 1 with an error message; evaluate prints a success rate; batch skips corrupt files and returns code 2.

Validation tests go beyond unit checks: the evaluation harness measures corner error against exact ground truth, which validates the whole pipeline numerically. **Negative tests** (bad config, corrupt files, degenerate geometry, noise images) confirm the error-handling strategy.

11. Challenges Faced

- **Hough fallback accuracy.** The first fallback picked the strongest lines by vote count and failed often: rows of text and table rules produce many strong, near-horizontal lines inside the page, outvoting the real borders. A second attempt scored every candidate quadrilateral by edge support but was worse (the true borders were not among the top-voted lines). The final design takes the *outermost* strong lines and adds an edge-support gate that rejects poor candidates. Lesson: measure, do not guess.
- **Misleading experiments.** Early tuning experiments called the fallback helpers directly at full resolution and suggested it was weak (under 50% success); evaluating it through the real pipeline path (downscaled processing, same parameters as deployed) gave 90%. The exact cause of the gap was not isolated, but the lesson is clear: evaluate a component exactly as it is deployed.
- **Aspect-ratio ambiguity.** A rectified page came out landscape although the source page was portrait. A single quadrilateral cannot determine the true aspect ratio without camera intrinsics; the optional A4/Letter prior was added as a practical remedy.
- **Uneven lighting.** Plain histogram equalisation amplified shadows. Dividing by an estimated background first (flat-field correction) solved this.
- **Numerical conditioning.** Un-normalised DLT is sensitive to coordinate scale; Hartley normalisation fixed it.
- **Evidence without a dataset.** No labelled document-photo set was available, so synthetic scenes with exact ground truth were generated. Their limits are stated openly.

12. Learnings & Key Takeaways

- Classical CV pipelines are chains of simple, testable steps; the quality of each step can be measured on its own.
- Projective geometry is concrete: four point pairs determine a homography, and the SVD solves it.
- Robustness comes from fallbacks, validation gates and honest failure modes, not from one clever algorithm.
- Ground-truth synthetic data enables ablation studies and repeatable evaluation.

- Good engineering (config validation, logging, exit codes, tests, incremental commits) is part of the result, not an afterthought.

13. Future Enhancements

- Evaluate on a public real-world dataset of document photos and report precision/recall.
- Sub-pixel corner refinement and robust line fitting with RANSAC for the fallback.
- Estimate the true aspect ratio from camera intrinsics or vanishing points (Unit 2: auto-calibration).
- Handle curved pages (dewarping) and multiple pages per frame.
- Add an OCR stage and report character error rate before vs. after enhancement.
- Add noise and blur robustness benchmarks; package for pip.

14. References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
- [2] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679-698, 1986.
- [3] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62-66, 1979.
- [4] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, no. 1, pp. 11-15, 1972.
- [5] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press, 1994, pp. 474-485.
- [6] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022.
- [7] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.
- [8] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000. <https://opencv.org>
- [9] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.